Document number: P0275R0
Project: Programming Language C++
Audience: Library Evolution Working Group

Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2016-03-20

A Proposal to add Classes and Functions Required for Dynamic Library Load

I. Introduction and Motivation

Adding a specific features to an existing software applications at runtime could be useful in many cases. Such extensions, or plugins, are usually implemented using Shared Library files (DLL, SO/DSO) loaded into the memory of program at runtime.

Current C++ Standard lacks support for dynamic library loading (DLL). This proposal attempts to fix that and provide a simple to use classes and functions for DLL.

A proof of concept implementation available at: Boost.DLL.

II. Impact on the Standard

This proposal is a pure library extension. It does not propose changes to existing headers. It relies on N4099, because it uses class experimental::path from that proposal. It does not require any changes in the core language and it could be implemented in standard C++.

III. Design Decisions

A. Usage of `std::experimental::filesystem::path`

To simplify library usage `std::experimental::filesystem::path` is used for specifying paths . All the `path` related overhead is minor, comparing to the time of loading shared library file into the memory of program or getting information about shared library file location.

B. Stick to the Filesystem error reporting scheme.

Provide two overloads for some functions, one that throws an exception to report system errors, and another that sets an `error_code`. This supports two common use cases:

- Uses where shared library file related system errors are truly exceptional and indicate a serious failure.
- Uses where shared library file related errors are routine and do not necessarily represent failure.

C. Do not take care of mangling.

C++ symbol mangling depend on compiler and platform. Existing attempts to get mangled symbol by unmangled name result in significant complexity and memory usage growth and overall slowdown of getting symbol from shared library file.

```
[Example:
    // Loading a shared library file with mangled symbols
    mangled_library lib("libmycpp.so");

    // Attempt to get function "foo::bar" with signature void(int)
    lib.get<void(int)>("foo::bar");

    // The problem is that we do not know what `foo::bar` is:
    // * `foo` could be a class and `bar` could be a static member function
    // * `foo` could be a struct and `bar` could be a static member function
    // * `foo` could be a namespace and `bar` could be function in that namespace
    //
    // We also do not know the calling convention for `foo` and it's noexcept specification.
    //
    // Mangling of `foo::bar` depends on all the knowledge from above, so we are either forced to
    // mangle and try to obtain all the available combinations; or we need to investigate all the
    // symbols exported by "libmycpp.so" shared library file and find a best match.
- end example]
```

While no good solution was found for obtaining mangled symbol by unmangled name, this proposal concentrates on obtaining symbols by exact name match.

D. Provide functions for simple usage.

Keeping a shared library file loaded in memory of program while using the symbol obtained from shared library file could be hard and error prone for some users. For the simplicity of those users, `import` functions were provided. Those functions return a symbol wrapped in a helper class along with an instance of shared_library. In that way while returned value in in scope, the shared library file won't be unloaded from program memory, so the user does not need to take care of shared library file lifetime in program memory.

E. Do not search libraries in system specific paths by default.

Searching paths for a specified shared library file may affect load times and make the proposed wording less efficient. It is assumed to be the most common case, that user exactly knows were the desired shared library file is located and provides either absolute or relative to current directory path. For that case searching system specific paths affects performance and increases the chance of finding wrong shared library file with the same name. Because some operating systems search system paths even if relative path is provided, the requirement to not do that is implicitly described in proposed wording.

F. Minimal modifications for the N4567 wording.

There have been a lot of shared library related proposals:

- N1400: Toward standardization of dynamic libraries
- N1418: Dynamic Libraries in C++
- N1428: Draft Proposal for Dynamic Libraries in C++
- N1496: Draft Proposal for Dynamic Libraries in C++ (Revision 1)
- N1976: Dynamic Shared Objects: Survey and Issues
- N2015: Plugins in C++
- N2407: C++ Dynamic Library Support

The proposal you're reading attempts to avoid issues of the proposals from above, minifying the changes to the N4567 wording. The proposal you're reading:

- is a pure library extension (does not modify Basic Concepts [basic] and does not add new keywords)
- does not add platform requiremnt for shared libraries suport

IV. Proposed wording relative to N4567

31 Dynamic library load support library [dll]

31.1 General [dll.general]

'Dynamic library load' is a runtime mechanism to load shared library file into the memory of current program, retrieve the addresses of functions and objects contained in the shared library file, execute those functions or access those objects, unload the shared library file from memory.

'Shared library file' is a file containing set of objects and functions available for use at program startup or at runtime.

Headers <experimental/dll> and <experimental/import> define classes and functions suitable for dynamic library load. For those headers term 'symbol' relates to a function, reference, class member or object that can be obtained from shared library file at runtime. Term 'symbol name' relates to a character identifier of a symbol, using which symbol can be obtained from shared library file. For symbols declared with extern "C" in code of shared library file, 'symbol name' is the name of the entity. 'symbol name' without extern "C" in C++ code of shared library file is the mangled name of the entity.

31.2 Error reporting [dll.errors]

Functions not having an argument of type error_code& report errors as follows, unless otherwise specified:

- When a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications, an exception of type system_error is thrown.
- Failure to allocate storage is reported by throwing an exception as described in the C++ standard, 17.6.4.10 [res.on.exception.handling].
- Destructors throw nothing.

Functions having an argument of type error_code& report errors as follows, unless otherwise specified:

- If a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications, the error_code& argument is set as appropriate for the specific error. Otherwise, clear() is called on the error_code& argument.
- Failure to allocate storage is reported by throwing an exception as described in the C++ standard, 17.6.4.10 [res.on.exception.handling].

31.3 Header <experimental/dll> [dll.dll]

```
namespace std {
  namespace experimental {
  inline namespace dll_v1 {

    // shared library file load modes
    enum class dll_mode {
        default_mode = 0,
        dont_resolve_dll_references,    // DONT_RESOLVE_DLL_REFERENCES
        load_ignore_code_authz_level,   // LOAD_IGNORE_CODE_AUTHZ_LEVEL
        rtld_lazy,                      // RTLD_LAZY
        rtld_now,                       // RTLD_NOW
        rtld_global,                    // RTLD_GLOBAL
        rtld_local,                     // RTLD_LOCAL
        rtld_deepbind,                  // RTLD_DEEPBIND
        append_decorations,             // See [dll.dll_mode]
        search_system_directories       // See [dll.dll_mode]
    };

    constexpr dll_mode  operator| (dll_mode lhs, dll_mode rhs)  noexcept;
    constexpr dll_mode& operator|=(dll_mode& lhs, dll_mode rhs) noexcept;
    constexpr dll_mode  operator& (dll_mode lhs, dll_mode rhs)  noexcept;
    constexpr dll_mode& operator&=(dll_mode& lhs, dll_mode rhs) noexcept;
    constexpr dll_mode  operator^ (dll_mode lhs, dll_mode rhs)  noexcept;
    constexpr dll_mode& operator^=(dll_mode& lhs, dll_mode rhs) noexcept;
    constexpr dll_mode& operator~ (dll_mode& lhs) noexcept;

    // class to work with shared library files
    class shared_library;

    bool operator==(const shared_library& lhs, const shared_library& rhs) noexcept;
    bool operator!=(const shared_library& lhs, const shared_library& rhs) noexcept;
    bool operator< (const shared_library& lhs, const shared_library& rhs) noexcept;
    bool operator> (const shared_library& lhs, const shared_library& rhs) noexcept;
    bool operator<=(const shared_library& lhs, const shared_library& rhs) noexcept;
    bool operator>=(const shared_library& lhs, const shared_library& rhs) noexcept;

    // free functions
    template<class T>
    filesystem::path symbol_location(const T& symbol, error_code& ec);
    template<class T>
    boost::filesystem::path symbol_location(const T& symbol);

    filesystem::path this_line_location(error_code& ec);
    filesystem::path this_line_location();

    filesystem::path program_location(error_code& ec);
    filesystem::path program_location();

  }
  }

  // support
  template <class T> struct hash;
  template <> struct hash<experimental::shared_library>;
  template <> struct hash<experimental::dll_mode>;
}
```

The value of each enum dll_mode constant shall be the same as the value of the macro shown in the above synopsis if that macro is available on the platform or 0 otherwise.

31.3.1 Enum dll_mode [dll.dll_mode]

Enum dll_mode provides modes for searching the shared library file and platform specific modes for loading the shared library file in memory of program. [Note: library users may extend available modes by casting the required platform specific mode to dll_mode: *[Example:* `static_cast<dll_mode>(RTLD_NODELETE);` *— end example] — end note]*

Each of system family provides own modes, flags not supported by a particular platform must be set to 0. Special modes are listened below:

`append_decorations`

Effects: Appends a platform specific extensions and prefixes to shared library filename before trying to load it into program memory. If load attempts fail, tries to load with exactly specified name.

Platforms: Windows, POSIX

Value: Any value that can not be received by applying binary OR to any set of modes from dll_mode

[Example:

```
        // Tries to open
        //      `./my_plugins/plugin1.dll` and `./my_plugins/libplugin1.dll` on Windows
        //      `./my_plugins/libplugin1.so` on Linux
        //      `./my_plugins/libplugin1.dylib` and `./my_plugins/libplugin1.so` on MacOS.
        // If that fails, loads `./my_plugins/plugin1`
        shared_library lib("./my_plugins/plugin1", dll_mode::append_decorations);
- end example]
```

`default_mode`

Effects: Default mode for a particular platform.

search_system_directories

Value: Any value that can not be received by applying binary OR to any set of modes from dll_mode

Effects: Allows loading of shared library files from system specific shared library file directories [fs.def.directory] along with loading shared library files from current directory.

31.3.1.1 dll_mode operators [dll.dll_mode.operators]

```
constexpr dll_mode  operator| (dll_mode lhs, dll_mode rhs)  noexcept;
constexpr dll_mode& operator|=(dll_mode& lhs, dll_mode rhs) noexcept;
constexpr dll_mode  operator& (dll_mode lhs, dll_mode rhs)  noexcept;
constexpr dll_mode& operator&=(dll_mode& lhs, dll_mode rhs) noexcept;
constexpr dll_mode  operator^ (dll_mode lhs, dll_mode rhs)  noexcept;
constexpr dll_mode& operator^=(dll_mode& lhs, dll_mode rhs) noexcept;
constexpr dll_mode& operator~ (dll_mode& lhs) noexcept;
```

Effects: Converts parameters to unsigned integral type capable of storing them, applies a corresponding binary operator and returns the result as dll_mode.

31.3.1.2 dll_mode hash support [dll.dll_mode.hash]

```
template <> struct hash<experimental::dll_mode>;
```

The template specialization shall meet the requirements of class template hash (20.9.13).

31.3.2 Class shared_library [dll.shared_library]

The class shared_library provides means to load shared library file into the memory of program, check that shared library file exports symbol with specified symbol name and obtain that symbol.

shared_library instances share reference count to an actual shared library file loaded in memory of current program, so it is safe and memory efficient to have multiple instances of shared_library referencing the same shared library file even if those instances were loaded in memory using different paths (relative and absolute) referencing the same shared library file.

It must be safe to concurrently load shared library files into memory of program, unload and get symbols from any shared library files using different shared_library instances. If current platform does not guarantee safe concurrent work with shared library files, calls to shared_library functions must be serialized by shared_library implementation.

[Note: Constructors, comparisons and reset() functions that accept nullptr_t are not provided because that may cause confusion: some of the platforms provide interfaces for shared library files that accept nullptr_t to get a handler to the current program. — end note]

```
namespace std {
  namespace experimental {
  inline namespace dll_v1 {

    class shared_library {
    public:
      typedef platform_specific native_handle_type;

      // construct/copy/destruct
      shared_library() noexcept;
      shared_library(const shared_library& lib);
      shared_library(const shared_library& lib, error_code& ec);
      shared_library(shared_library&& lib) noexcept;
      explicit shared_library(const filesystem::path& library_path, dll_mode mode = dll_mode::default_mode);
      shared_library(const filesystem::path& library_path, error_code& ec, dll_mode mode = dll_mode::default_mode);
      shared_library(const filesystem::path& library_path, dll_mode mode, error_code& ec);
      ~shared_library();

      // public member functions
      shared_library& assign(const shared_library& lib, error_code& ec);
      shared_library& assign(const shared_library& lib);

      shared_library& operator=(const shared_library& lib);
      shared_library& operator=(shared_library&& lib) noexcept;

      void load(const filesystem::path& library_path, dll_mode mode = dll_mode::default_mode);
      void load(const filesystem::path& library_path, error_code& ec, dll_mode mode = dll_mode::default_mode);
      void load(const filesystem::path& library_path, dll_mode mode, error_code& ec);

      void reset() noexcept;
      explicit operator bool() const noexcept;

      bool has(const char* symbol_name) const noexcept;
      bool has(const string& symbol_name) const noexcept;

      template <typename SymbolT>
      auto get(const char* symbol_name) const
        -> conditional_t<is_member_pointer_v<SymbolT>, SymbolT, add_lvalue_reference_t<SymbolT>>;

      template <typename SymbolT>
      auto get(const string& symbol_name) const
        -> conditional_t<is_member_pointer_v<SymbolT>, SymbolT, add_lvalue_reference_t<SymbolT>>;

      native_handle_type native_handle() const noexcept;

      filesystem::path location() const;
      filesystem::path location(error_code& ec) const;
```

```
      // public static member functions
      static constexpr bool platform_supports_dll() noexcept;
      static constexpr bool platform_supports_dll_of_program() noexcept;
    };

  }
  }
}
```

31.3.3.1 shared_library constructors [dll.shared_library.constr]

```
shared_library() noexcept;
```

Effects: Creates shared_library that does not reference any shared library file.

Postconditions: *this is false.

```
shared_library(const shared_library& lib);
shared_library(const shared_library& lib, error_code& ec);
```

Effects: Makes *this reference the same shared library file that is referenced by lib or if lib does not reference shared library file sets *this to default constructed state.

Postconditions: lib == *this

Throws: As specified in [dll.errors]

```
shared_library(shared_library&& lib) noexcept;
```

Effects: Assigns the state of lib to *this and sets lib to a default constructed state.

Remarks: Does not invalidate symbols previously obtained from lib.

Postconditions: lib is false, *this is true.

```
explicit shared_library(const filesystem::path& library_path, dll_mode mode = dll_mode::default_mode);
shared_library(const filesystem::path& library_path, error_code& ec, dll_mode mode = dll_mode::default_mode);
shared_library(const filesystem::path& library_path, dll_mode mode, error_code& ec);
```

Effects: Same as calling shared_library::load() with same parameters.

Throws: As specified in [dll.errors].

31.3.3.2 shared_library destructor [dll.shared_library.destr]

```
~shared_library();
```

Effects: Destroys the shared_library by calling reset(). If the shared library file is referenced by different instances of shared_library, the shared library file won't be unloaded until there is at least one instance of shared_library referencing it.

31.3.3.3 shared_library members [dll.shared_library.member]

```
shared_library& assign(const shared_library& lib, error_code& ec);
shared_library& assign(const shared_library& lib);
```

Effects: If *this, then calls reset(). Makes *this reference the same shared library file as lib.

Postconditions: lib.location() == this->location(), lib == *this

Throws: As specified in [dll.errors]

31.3.3.4 shared_library assignment [dll.shared_library.assign]

```
shared_library& operator=(const shared_library& lib);
```

Effects: Same as calling this->assign(lib)

```
shared_library& operator=(shared_library&& lib) noexcept;
```

Effects: If *this then calls this->reset(). Assigns the state of lib to *this and sets lib to a default constructed state.

Remarks: Does not invalidate symbols previously obtained from lib.

Postconditions: lib is false, *this is true.

```
void load(const filesystem::path& library_path, dll_mode mode = dll_mode::default_mode);
void load(const filesystem::path& library_path, error_code& ec, dll_mode mode = dll_mode::default_mode);
void load(const filesystem::path& library_path, dll_mode mode, error_code& ec);
```

Effects: If the shared library file specified by library_path is already loaded in memory of current program makes *this reference the previously loaded shared library file. Otherwise loads a shared library file by specified path with a specified mode into the memory of current program, executes all the platform specific initializations and makes the symbols of that shared library file ready to obtain using this->get(). If some shared library file is already referenced by *this, calls reset() first.

Must be capable of referencing current program if absolute or relative path to the program was provided as library_path. Must try to load shared library file with changed name and applied extension only if (mode & dll_mode::append_decorations) is not 0. Must attempt to load a shared library file from system specific shared library file directories only if library_path contains only shared library filename and (mode & dll_mode::search_system_directories) is not 0. During loads from system specific directories file name modification rules from above apply. If with dll_mode::search_system_directories more than one shared library file has the same base name and extension, the function loads in memory any first matching shared library file. If (mode & dll_mode::search_system_directories) is 0, then any relative path or path that contains only filename must be treated as a path in current working directory.

Library open mode is equal to (mode & ~dll_mode::search_system_directories & ~dll_mode::append_decorations) converted to a platform specific type representing shared library file load modes and adjusted to satisfy the dll_mode::search_system_directories requirements from above. If mode is invalid for current platform, attempts to adjust the mode by applying dll_mode::rtld_lazy and throws if resulting mode is invalid. *[Example:* If mode on POSIX was set to rtld_local, then it will be adjusted to rtld_lazy | rtld_local. - end example]

Throws: As specified in [dll.errors]

```
void reset() noexcept;
```

Effects: Sets *this to a default constructed state. If *this was the last instance of shared_library referencing the shared library file, then all the resources associated with shared library file shall be released.

Remarks: Symbols obtained from shared library file remain valid until there is at least one instance of shared_library referencing the shared library file.

Postconditions: *this is false.

```
explicit operator bool() const noexcept;
```

Returns: true if *this references a shared library file.

```
bool has(const char* symbol_name) const noexcept;
bool has(const string& symbol_name) const noexcept;
```

Returns: true if *this references a shared library file and symbol symbol_name could be obtained from shared library file.

```
template <typename SymbolT>
auto get(const char* symbol_name) const
    -> conditional_t<is_member_pointer_v<SymbolT>, SymbolT, add_lvalue_reference_t<SymbolT>>;

template <typename SymbolT>
auto get(const string& symbol_name) const
    -> conditional_t<is_member_pointer_v<SymbolT>, SymbolT, add_lvalue_reference_t<SymbolT>>;
```

Returns: Symbol from shared library file that has the name symbol_name.

Remarks: It's the user responsibility to provide valid SymbolT type for symbol. However implementations may provide additional checks for matching SymbolT type and actual symbol type. [Note: For example implementations may check that SymbolT is a function pointer and that symbol with symbol_name allows execution. — end note]

Throws: system_error if symbol does not exist or if the shared library file was not loaded.

```
native_handle_type native_handle() const noexcept;
```

Returns: Native handler of the loaded in memory shared library file or default constructed native_handle_type if *this does not reference a shared library file. [Note: This member allow implementations to provide access to implementation details. Actual use of these members is inherently non-portable. — end note]

```
filesystem::path location() const;
filesystem::path location(error_code& ec) const;
```

Returns: Full path and name to the referenced shared library file, throws if *this does not reference shared library file.

Throws: As specified in [dll.errors]

31.3.3.5 shared_library static members [dll.shared_library.static]

```
static constexpr bool platform_supports_dll() noexcept;
```

Returns: true if platform supports loading of shared library files into the momery of program. [Note: If this function returns false, then any attempt to load a shared library file will fail at runtime. This function could be used to ensure platform capabilities at compile time: `static_assert(shared_library::platform_supports_dll(), "DLL required for this program");` — end note]

```
static constexpr bool platform_supports_dll_of_program() noexcept
```

Returns: true if according to platform capabilities `shared_library(program_location())` may succeed.

31.3.3.6 shared_library free operators [dll.shared_library.operators]

shared_library provides fast comparison operators that compare the referenced shared libraries file.

```
bool operator==(const shared_library& lhs, const shared_library& rhs) noexcept;
```

Returns: lhs.native_handle() == rhs.native_handle()

```
bool operator!=(const shared_library& lhs, const shared_library& rhs) noexcept;
```

Returns: lhs.native_handle() != rhs.native_handle()

```
bool operator<(const shared_library& lhs, const shared_library& rhs) noexcept;
```

Returns: lhs.native_handle() < rhs.native_handle()

```
bool operator>(const shared_library& lhs, const shared_library& rhs) noexcept;
```

Returns: lhs.native_handle() > rhs.native_handle()

```
bool operator<=(const shared_library& lhs, const shared_library& rhs) noexcept;
```

Returns: lhs.native_handle() <= rhs.native_handle()

```
bool operator>=(const shared_library& lhs, const shared_library& rhs) noexcept;
```

Returns: lhs.native_handle() >= rhs.native_handle()

31.3.3.7 shared_library hash support [dll.shared_library.hash]

```
template <> struct hash<experimental::shared_library>;
```

The template specialization shall meet the requirements of class template hash (20.9.13).

31.3.4 Runtime path functions [dll.location]

```
template<class T>
filesystem::path symbol_location(const T& symbol, error_code& ec);
template<class T>
boost::filesystem::path symbol_location(const T& symbol);
```

Returns: Full path and name to shared library file or program that contains symbol

Throws: As specified in [dll.errors]

```
filesystem::path this_line_location(error_code& ec);
filesystem::path this_line_location();
```

Returns: Full path and name to shared library file or program that contains line of code in which this_line_location() was called

Throws: As specified in [dll.errors]

```
filesystem::path program_location(error_code& ec);
filesystem::path program_location();
```

Returns: Full path and name to the current program.

Throws: As specified in [dll.errors]

31.4 Header <experimental/import> [dll.import.header]

```
#include <experimental/dll>
#include <type_traits>
#include <memory>
#include <functional>

namespace std {
  namespace experimental {
  inline namespace dll_v1 {

    template <typename SymbolT>
    using imported_t = conditional_t<is_object_v<SymbolT>, shared_ptr<SymbolT>, function<SymbolT>>;

    // functions for importing a symbol
    template <typename SymbolT>
    imported_t<SymbolT> import(const filesystem::path& library_path, const char* symbol_name, dll_mode mode = dll_mode::default_mode);

    template <typename SymbolT>
    imported_t<SymbolT> import(const filesystem::path& library_path, const string& symbol_name, dll_mode mode = dll_mode::default_mode);

    template <typename SymbolT>
    imported_t<SymbolT> import(const shared_library& library, const char* symbol_name);

    template <typename SymbolT>
    imported_t<SymbolT> import(const shared_library& library, const string& symbol_name);

    template <typename SymbolT>
    imported_t<SymbolT> import(shared_library&& library, const char* symbol_name);
```

```
    template <typename SymbolT>
    imported_t<SymbolT> import(shared_library&& library, const string& symbol_name);

  }
  }
}
```

import functions are meant to simplify dynamic library loads of symbols by keeping shared library file loaded in program memory while imported symbol is in scope:

```
[Example:
    // Code of "/plugin_directory/libplugin.so" contains extern "C" void foo_function(string&&)

    auto foo_function = import<void(string&&)>("/plugin_directory/libplugin.so", "foo_function");
    foo_function("Test");
- end example]
```

31.4.1 import functions **[dll.import.func]**

```
template <typename SymbolT>
imported_t<SymbolT> import(const filesystem::path& library_path, const char* symbol_name, dll_mode mode = dll_mode::default_mode);

template <typename SymbolT>
imported_t<SymbolT> import(const filesystem::path& library_path, const string& symbol_name, dll_mode mode = dll_mode::default_mode);

template <typename SymbolT>
imported_t<SymbolT> import(const shared_library& library, const char* symbol_name);

template <typename SymbolT>
imported_t<SymbolT> import(const shared_library& library, const string& symbol_name);

template <typename SymbolT>
imported_t<SymbolT> import(shared_library&& library, const char* symbol_name);

template <typename SymbolT>
imported_t<SymbolT> import(shared_library&& library, const string& symbol_name);
```

> *Effects:* Obtains a symbol with name symbol_name from shared library file and returns it wrapped in class that implements semantics of shared ownership of shared library file; the last remaining owner is responsible for releasing the resources associated with the shared library file. mode must be passed to shared_library constructor or shared_library::load function.
>
> *Remarks:* It's the user responsibility to provide valid SymbolT type for symbol.
>
> *Return type:* imported_t<SymbolT> that keeps an instance an of shared_library internally
>
> *Returns:* Variable that keeps shared_library and symbol and provides access to the symbol only.
>
> *Throws:* system_error if symbol does not exist or if failed to load the shared library file into the memory of program.

V. Feature-testing macro

For the purposes of SG10 it is sufficient to check for header <experimental/dll> or <experimental/import> using __has_include.

V. Revision History

Revision 0:

- Initial proposal

VI. References

[Boost.DLL] Boost DLL library. Available online at https://github.com/boostorg/dll

[N4567] Working Draft, Standard for Programming Language C++. Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4567.pdf

[N4099] Working Draft, Technical Specification — File System. Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4099.html

[N1400] Toward standardization of dynamic libraries. Available online at http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2002/n1400.html

[N1418] Dynamic Libraries in C++. Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2002/n1418.html

[N1428] Draft Proposal for Dynamic Libraries in C++. Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2003/n1428.html

[N1496] Draft Proposal for Dynamic Libraries in C++ (Revision 1). Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2003/n1496.html

[N1976] Dynamic Shared Objects: Survey and Issues. Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2006/n1976.html

[N2015] Plugins in C++. Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2006/n2015.pdf

[N2407] C++ Dynamic Library Support. Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2407.html

VII. Acknowledgements

Klemens Morgenstern highlighted some of the missing functionality in Boost.DLL and provided implementation of mangled symbols load, that showed complexities of such approach. Renato Tegon Forti started the work on Boost.DLL and provided a lot of code, help and documentation for the Boost.DLL.